
SDLC – Unit Breakdown & AI Prompts (PHP MVC Project)

How to Use This Document

1. Work on **one unit at a time** in the order below.
 2. For each unit, copy the  **Prompt** into your AI tool.
 3. Review, test, and integrate the generated code before moving to the next unit.
 4. Each unit is independent – you can vibe code in isolation.
-

Unit 0 – Database Schema (SQL)

Responsibility: Define tables, columns, foreign keys (min. 3 related tables).

Dependencies: None – create first.

Output: `database.sql` file.

Prompt

You are an expert SQL designer. Create a MySQL schema for a PHP MVC project with at least 3 related tables (e.g., users, products, categories, or posts, comments, authors). Include:

- Primary keys, auto-increment
- Foreign key constraints with ON DELETE CASCADE where appropriate
- Sample data (3-5 rows per table)
- Use InnoDB, UTF-8 charset

Output only the SQL code, no explanation. Name the file `database.sql`.

Unit 1 – Database Singleton (PDO)

Responsibility: Single PDO connection instance.

Depends on: Unit 0 (schema exists).

Output: `Database.php`

Prompt

Write a PHP class called Database that implements the Singleton pattern. It should:

- Use PDO for MySQL

- Read credentials from a config array (host, dbname, user, password)
- Set PDO options: exceptions on error, persistent connection, fetch mode associative by default
- Provide a public static method getInstance() returning the unique PDO instance
- Prevent cloning and wakeup

Write clean, commented code. No HTML. Only the class.

✅ Unit 2 – Entity Classes (POPO)

Responsibility: One plain PHP class per table (properties + getters/setters).

Depends on: Unit 0 (table structure).

Output: User.php, Product.php, Category.php (or your table names).

Prompt

Generate PHP entity classes (Plain Old PHP Objects) for each table defined in this SQL schema:

[PASTE YOUR database.sql HERE]

For each table, create a class with:

- Private properties matching columns (camelCase)
- Constructor that optionally accepts an associative array to hydrate
- Getters and setters for each property
- No database logic – purely for data representation

Example format shown for User. Return only the code for all entity classes in separate files.

✅ Unit 3 – DAO / Manager Classes (CRUD)

Responsibility: One DAO per entity with save(), find(), findAll(), delete(), update().

Depends on: Unit 1 (Database) + Unit 2 (Entities).

Output: UserDao.php, ProductDao.php, etc.

Prompt

Write a Data Access Object (DAO) for a table named [TABLE_NAME]. Use the Database singleton from Unit 1. The DAO must:

- Accept a PDO instance via constructor (use Database::getInstance())

- Implement:
 - find(\$id): returns an entity object or null
 - findAll(): returns array of entity objects
 - save(\$entity): inserts if no id, updates if exists - returns bool/ID
 - delete(\$id): returns bool
- Use prepared statements
- Assume entity class exists with getters/setters matching column names

Write only the DAO code for [TABLE_NAME]. Include use statements. No HTML.

Repeat this prompt for each table.

✅ Unit 4 – Simple Router & Front Controller

Responsibility: Parse URL/query parameters and call controller actions.

Depends on: None directly (calls controllers later).

Output: index.php (in /public folder) and Router.php .

📌 Prompt

Create a minimal PHP router for an MVC application. The router should:

- Read the request from \$_GET['route'] or \$_SERVER['REQUEST_URI'] (e.g., "/user/show/5")
- Parse it into controller name, action, and parameters
- Map to a controller class file in /Controllers
- Instantiate the controller and call the action method, passing parameters
- Handle 404 if controller or action not found

Provide two files:

1. Router.php - class with a dispatch() method
2. index.php - front controller that includes Router.php and runs dispatch()

Assume URL format: index.php?route=user/show/5 or clean URL /user/show/5.
Write code with comments.

✅ Unit 5 – Authentication Module

Responsibility: Login, logout, session protection, password verification.

Depends on: Unit 3 (UserDao) + Unit 4 (Router).

Output: AuthController.php , Auth.php (helper), login/logout views.

📌 Prompt

Build a complete authentication module for PHP MVC:

- AuthController with actions: login(), logout(), dashboard()
- A helper class Auth with static methods: isLoggedIn(), requireLogin(), login(\$user), logout()
- Use \$_SESSION, regenerate session ID on login
- Passwords stored with password_hash() and verified with password_verify()
- Login view: simple HTML form that POSTs to AuthController@login
- Protected pages redirect to login if not authenticated
- After login, redirect to dashboard

Write all code in separate files with clear names. Use the UserDao from Unit 3 to find users by email/username. No hardcoded credentials.

Unit 6 – CRUD Controllers & Views (One Entity at a Time)

Responsibility: Standard CRUD UI for each main entity.

Depends on: Unit 3 (DAO) + Unit 4 (Router) + Unit 5 (Auth protection).

Output: ProductController.php, list.php, form.php, show.php views.

Prompt

Generate a full CRUD controller and views for an entity called [ENTITY_NAME] (e.g., Product).

- Controller actions: index (list), show (detail), create (show form), store (save new), edit (show edit form), update, delete
- Each action must check authentication (Auth::requireLogin())
- Use the corresponding DAO for database operations
- Views: plain PHP/HTML with a layout (header/footer)
- Forms POST to the same controller's store/update actions
- Delete via POST with CSRF protection (simple token)

Write code for:

- [ENTITY_NAME]Controller.php
- views/[entity]/index.php, show.php, create.php, edit.php

Assume the entity has standard fields (name, description, price, etc.). Return clean, organized code.

Unit 7 – Uploader Utility

Responsibility: Secure file upload handling (validation, rename, move).

Depends on: None (standalone utility).

Output: Uploader.php

Prompt

Write a PHP class Uploader with static methods:

- upload(\$file, \$targetDir, \$maxSize = 2MB, \$allowedTypes = ['jpg', 'png', 'pdf'])
- Generates a unique name (timestamp + random string)
- Validates: file exists, no error, size, extension, MIME type
- Moves file to target directory, returns new filename or throws exception
- Optional: create directory if not exists

Include error constants. No database logic. Use clean OOP.

Unit 8 – Admin File Management UI

Responsibility: List uploaded files, upload new, delete.

Depends on: Unit 7 (Uploader) + Unit 5 (Auth) + simple DAO for file metadata.

Output: FileManagerController.php, upload_form.php, file_list.php, FileDao.php (optional).

Prompt

Create an admin file management module:

- Database table 'files' with: id, original_name, stored_name, mime_type, size, uploaded_at, user_id (FK to users)
- FileDao with methods: saveFileRecord(), findAll(), deleteById()
- FileManagerController with actions: index (list files), upload (POST), delete (POST)
- Upload action uses Uploader class from Unit 7, then saves record to database
- Index view displays list of uploaded files with download/delete links
- Protect all actions with Auth::requireLogin() and check admin role (optional: is_admin column)

Generate: SQL for 'files' table, FileDao.php, FileManagerController.php, and two views. Assume UserDao exists.

Unit 9 – Search Module (Multi-Criteria)

Responsibility: Search across multiple tables with filters.

Depends on: Unit 1 (Database) + Unit 3 (DAOs).

Output: SearchController.php, search_view.php, SearchService.php.

Prompt

Build a search module for the application:

- SearchController with action search() that reads GET parameters (keywords, category, price_min, price_max, etc.)
- SearchService class that uses raw PDO (or existing DAOs) to build a dynamic query with prepared statements
- Search results displayed in a clean table with links to each entity's detail page
- Support pagination (limit/offset)
- Allow search in at least two related tables (e.g., products and categories)

Write SearchController, SearchService, and a search form view. Queries must be injection-safe. Return only code.

Unit 10 – Documentation Files

Responsibility: README, user guide, AI report.

Depends on: None (final).

Output: README.md, user_guide.pdf, ai_report.pdf.

Prompt (for README)

Generate a professional README.md for a PHP MVC project. Include:

- Project title and description
- Installation steps (clone, configure database, run migrations, set docroot to /public)
- Requirements (PHP version, MySQL, extensions)
- Configuration instructions (copy config.example.php to config.php)
- Default admin credentials
- List of features (CRUD, auth, file upload, search)
- Folder structure

Write in Markdown, ready to copy-paste.

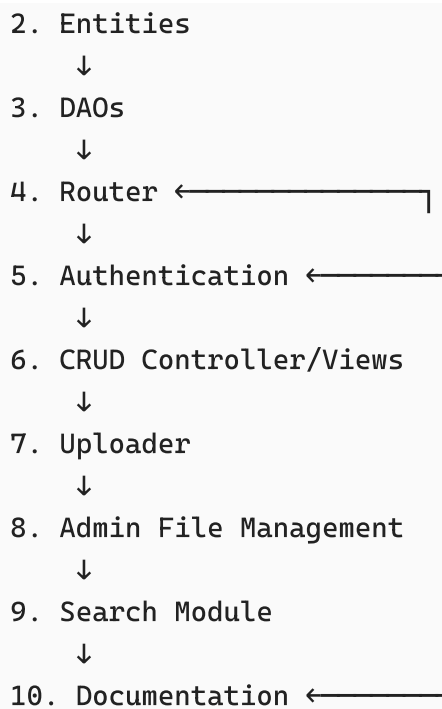
Unit Dependency Map (for your reference)

0. SQL Schema

↓

1. Database Singleton

↓



Final Advice for Vibe Coding

-  Test each unit immediately after generation (create a small `test.php` that includes the unit).
-  Keep a **simple integration plan**: after units 1–4 are done, you already have a working router + DB.
-  If an AI gives you code you don't understand, ask it to explain in simple terms **before** pasting.
-  Use **file naming conventions** exactly as the prompts suggest to avoid confusion.

Would you like me to also provide the **integration test script** for the first 4 units? That would let you verify everything works before adding more features.